SHRINCS specification

Blockstream Research

January 22, 2026

Abstract

This document specifies **SHRINCS** (Stateful Hash-based Robust Integrated NIST-Compatible Signatures), a post-quantum signature scheme optimized for Bitcoin. SHRINCS combines efficient stateful hash-based signatures (unbalanced XMSS with WOTS+C) for normal operation with a stateless fallback (SPHINCS+ variant) for scenarios when state management failed. The scheme achieves minimal 324-byte signatures during normal stateful operation while maintaining the robustness of stateless signatures as a backup when state is corrupted or lost.

Contents

1 Motivation **3**

2 Notation **3**

3 Overview **3**
3.1 Key Pair Derivation and Size . . . 4
3.2 Signature Size and Verification Cost . . . 5
3.2.1 Stateful signature size . . . 5

4 Parameters **5**

5 Underlying Functions and Addresses **5**
5.1 Hash Function . . . 5
5.2 Tweakable Hash Function . . . 5
5.3 Pseudorandom Functions . . . 6
5.4 Message Hash Function . . . 6
5.5 Tweak Structure and Domain Separation . . . 6
5.5.1 Address Types . . . 6
5.5.2 Address Manipulation Functions . . . 7
5.5.3 Type-Dependent Words . . . 7

6 WOTS+C **7**
6.1 The Target Condition . . . 8
6.2 Base-w Representation . . . 8
6.3 Chain Function . . . 9
6.4 Key Generation . . . 10
6.5 Messsage Digest Grinding . . . 10
6.6 Signature Generation . . . 11
6.7 Signature Verification (Public Key Recovery) . . . 12

7 UXMSS. The Stateful Tree **12**
7.1 UXMSS Tree Structure . . . 13
7.2 UXMSS Authentication Path . . . 13

8 FORS+C **14**
8.0.1 FORS Structure . . . 14
8.0.2 Parameters . . . 14
8.0.3 Grinding Condition . . . 14
8.0.4 Message to Indices . . . 14
8.0.5 Grinding . . . 15
8.0.6 Secret Key Generation . . . 15
8.0.7 TreeHash . . . 15
8.0.8 FORS Authentication path . . . 16
8.0.9 Signature generation . . . 16
8.0.10 Public Key Recovery . . . 17

9 SHRINCS **18**
9.0.1 Key Generation . . . 18
9.0.2 Recovery from Seed . . . 18
9.1 Statefull signing . . . 19
9.2 Stateless signing . . . 19
9.3 Statefull verification . . . 19
9.4 Stateless verification . . . 19

10 Data structures **19**
10.1 Secret Key Format . . . 19
10.2 Public Key Format . . . 19
10.3 Stateful Signature Format . . . 19
10.4 Stateless Signature Format . . . 19

1 Motivation

Current Bitcoin signature schemes (ECDSA and Schnorr over secp256k1) are vulnerable to quantum attacks via Shor's algorithm. Hash-based signature schemes offer post-quantum security relying only on the security of cryptographic hash functions, which are believed to resist quantum attacks apart from Grover's quadratic speedup.

The key challenges for deploying hash-based signatures in Bitcoin are:

1. Signature size: NIST-standardized SLH-DSA signatures range from 7KB to 50KB, significantly larger than current 64-byte Schnorr signatures.
2. Stateful vs. Stateless tradeoff: Stateful schemes (XMSS) offer small signatures but require careful state management. Stateless schemes (SLH-DSA) are robust but produce large signatures.
3. Backup compatibility: Bitcoin users expect to restore wallets from seed phrases. Pure stateful schemes break this model since state cannot be statically backed up.

SHRINCS addresses these challenges by:

- Using efficient stateful signatures ($\approx$ 324 bytes for $q = 1$) during normal operation.
- Falling back to stateless signatures ($\approx$ 3-4 KB) when state is lost or corrupted.
- Allowing static seed backup while maintaining security guarantees.

2 Notation

Throughout this specification:

`||` denotes byte string concatenation

`[a:b]` denotes a substring from byte index `a` to `(b-1)`

`[0:a]` denotes first `a` bytes (truncation)

`←$` uniform random sampling

`n` is the security parameter in bytes

`H` denotes a cryptographic hash function

`Tw` denotes a tweakable hash function

`PRF` denotes a pseudorandom function

$\perp$ used for failure returns

3 Overview

SHRINCS is designed in the way it can operate in two modes: **stateful** and **stateless**. Stateful mode is represented by the instance of Unbalanced XMSS (UXMSS) with a root public key `PK.sf`. An optimized variant of SPHINCS+ is used to support the stateless operating mode (which is represented by the public key `PK.sl`). The root public key is a hashed concatenation of (`PK.sf`, `PK.sl`), which is being converted to the Bitcoin address.

During normal operation with intact state, the user generates signatures according to efficient stateful path. Each leaf of the UXMSS is represented by WOTS+C one time signature instance. UXMSS is right-skewed, which makes the first stateful signature (`q=1`) the cheapest from the verification and size perspective (324 bytes) and adds the constant size (16 bytes) and one additional hashing operation for each next stateful signature. A right-skewed UXMSS is a tree construction in which every internal node has a one-time signature leaf as its left child and a UXMSS subtree as its right child, yielding a recursively right-branching authentication tree (as shown on the Figure 1).

If the state is corrupted (or the limit for stateful operations is met), the signature can continue operating with stateless mode. Stateless mode utilizes an optimized SLH-DSA variant with WOTS+ signatures in regular XMSS trees on each layer in the hypertree and FORS+C for few-time signature scheme. The general construction of SHRINCS signature is presented on the Figure 1.

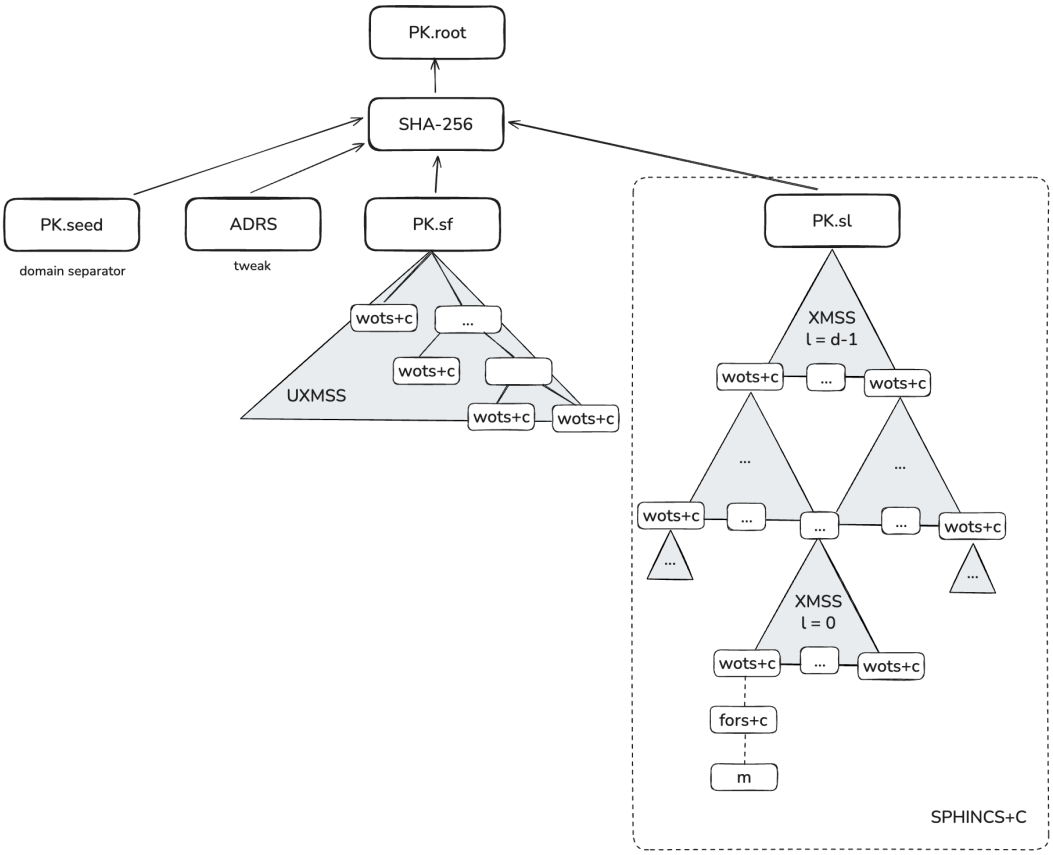

Figure 1: SHRINCS architecture

3.1 Key Pair Derivation and Size

SHRINCS keys are derived from a single random seed (**Oleks:** add assumptions about BIP-39 usage):

```
seed ←$ {0,1}^(3n)
```

Seed allows to derive a private key `SK ← (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)`:

```
SK.seed ← seed[0:n]  // Seed for key derivation
SK.prf ← seed[n:2n]  // Secret needed for PRF_msg
PK.seed ← seed[2n:3n] // Domain separator for different SHRINCS key pairs
PK.sf ← compute_stateful_root(PK.seed, ADRS, SK.seed) // Public stateful key, see Section X
PK.sl ← compute_stateless_root(PK.seed, ADRS, SK.seed) // Public stateless key, see Section X
PK.root ← H(PK.seed, ADRS, PK.sf, PK.sl) // Root public key
```

where `ADRS` is a context address (see Section 5.5).

The size of the secret key is `6n` bytes. A public key consists of two values `PK ← (PK.seed, PK.root)` and has a size of `2n` bytes.

3.2 Signature Size and Verification Cost

3.2.1 Stateful signature size

The WOTS+C component is 292 bytes: randomness (32 bytes) + counter (4 bytes) + chain values (256 bytes). The authentication path for signature `q` (count after signing) is `q * n = q * 16` bytes. The stateless public key `PK.sl` adds 16 bytes.

`sig.sf = 292 + q * 16 + 16 = 308 + q * 16` bytes,

where `q` is the signature count after signing:

q	Auth path	Total
1	16 bytes	324 bytes
2	32 bytes	340 bytes
3	48 bytes	356 bytes
10	160 bytes	468 bytes

4 Parameters

This specification provides with the list of parameters for different SHRINCS instances, depending on signer and verifier requirements:

1. SHRINCS-B: size matters (instance preferable for Bitcoin).
2. SHRINCS-L: verification complexity matters (the number of hashes to verify the signature).
3. SHRINCS-C: client-side efficiency matters. **Oleks:** todo if makes sense

Parameter	SHRINCS-B	SHRINCS-L	Description
`H`	`SHA-256`	`SHA-256`	Hash function
`OTS`	`WOTS+C`	`WOTS+C`	One-time signature
`FTS`	`FORS+C`	`FORS+C`	Few-time signature
`n`	16	16	Security parameter (bytes)
`w`	256	4	Winternitz parameter
`l`	16	64	The number of WOTS+C chains
`Swn`	2040	140	Target sum for WOTS+C
`hsf`	15	15	Maximum stateful tree height
`hsl`	40	40	Maximum stateless hypertree height
`d`	5	5	Stateless hypertree layers
`a`	22	22	FORS tree height
`k`	6	6	Number of FORS+C trees
`c`	4	4	Counter size for WOTS+C grinding (bytes)

5 Underlying Functions and Addresses

5.1 Hash Function

SHRINCS uses SHA-256 truncated to `n` bytes for all internal hash operations.

```
H(m) = SHA-256(m)[0:n]
```

5.2 Tweakable Hash Function

To prevent multi-target attacks and ensure domain separation between the various components, we employ the tweakable hash function construction defined in FIPS 205 [**fips-205**]:

```
Tw(PK.seed, ADRS, m) = H(PK.seed || ADRS || m)
```

This usage of tweaked hashing is critical. It ensures that a hash computed at one position in the Merkle tree cannot be substituted for a hash at another position, effectively mitigating generic tree-grafting attacks.

5.3 Pseudorandom Functions

We use PRF for deterministic key derivation:

```
PRF(PK.seed, ADRS, SK.seed) = Tw(PK.seed, ADRS, SK.seed)
```

Additionally we use a pseudorandom function to generate the randomizer `R` for the randomized hashing of the message to be signed.

```
PRF_msg(SK.prf, PK.seed, opt_rand, m) = HMAC-SHA-256(SK.prf, PK.seed || opt_rand || m),
```

where `opt_rand` is an optional randomness (`n` bytes), can be `PK.seed` for deterministic signing.

5.4 Message Hash Function

To generate the digest of the message to be signed we use the following function:

```
H_msg(ADRS, R, PK.seed, PK.root, m) = H(ADRS || R || PK.seed || PK.root || m)
```

5.5 Tweak Structure and Domain Separation

Tweaks are structured as follows to ensure uniqueness across all scheme components:

```
ADRS = layer || tree_address || type || words
```

where:

`layer`: (4 bytes) layer in hypertree (`0` = bottom, `d-1` = max)

`tree_address`: (12 bytes) tree address within layer

`type`: (4 bytes) address type

`words`: (12 bytes) type-dependent 12 bytes

A total `ADRS` size is 32 bytes.

5.5.1 Address Types

`0x00`: Stateful WOTS+C chains (`SF_WOTS_CHAIN`)

`0x01`: Stateful tree nodes (`SF_TREE`)

`0x02`: Stateless WOTS+C chains (`SL_WOTS_CHAIN`)

`0x03`: Stateless tree nodes (`SL_TREE`)

`0x04`: FORS (`SL_FORS`)

`0x05`: Message hash (`H_MSG`)

`0x06`: Pseudorandom function (`PRF`)

`0x07`: Root public key (`ROOT`)

Summarizing, `(0x00, 0x01)` are stateful only types, `(0x02, 0x03, 0x04)` are stateless only and `(0x05, 0x06, 0x07)` are shared types.

5.5.2 Address Manipulation Functions

```
setLayerAddress(ADRS, layer):
    ADRS[0:4] ← toByte(layer, 4)

setTreeAddress(ADRS, tree_addr):
    ADRS[4:16] ← toByte(tree_addr, 12)

setTypeAndClear(ADRS, type):
    ADRS[16:20] ← toByte(type, 4)
    ADRS[20:32] ← toByte(0, 12)

setKeyPairAddress(ADRS, keypair):
    ADRS[20:24] ← toByte(keypair, 4)

setChainAddress(ADRS, chain):
    ADRS[24:28] ← toByte(chain, 4)

setHashAddress(ADRS, hash):
    ADRS[28:32] ← toByte(hash, 4)

setTreeHeight(ADRS, height):
    ADRS[24:28] ← toByte(height, 4)

setTreeIndex(ADRS, index):
    ADRS[28:32] ← toByte(index, 4)
```

Note that `setTreeHeight` and `setChainAddress` both write to bytes `[24:28]`. These operations are context-dependent so there mustn't be a conflict.

5.5.3 Type-Dependent Words

Type	Meaning	word1 [20:24]	word2 [24:28]	word3 [28:32]
0x00	SF_WOTS_CHAIN	keypair	chain	hash
0x01	SF_TREE	keypair	height	index
0x02	SL_WOTS_CHAIN	keypair	chain	hash
0x03	SL_TREE	keypair	height	index
0x04	SL_FORS	tree_idx	height	index
0x05	H_MSG	0	0	0
0x06	PRF	W: keypair / F: tree_idx	W: chain / F: 0	W: 0 / F: leaf_idx
0x07	ROOT	0	0	0

Table 1: Type-dependent interpretation of ADRS words (bytes [20:32]). W is a WOTS an F is a FORS

6 WOTS+C

The standard Winternitz One-Time Signature (WOTS+) is the building block of most modern hash-based schemes. It works by encoding the message into a series of small integers and using them to iterate hash chains. A significant portion of a WOTS+ signature size (approx 20%) is dedicated to "Checksum," which is necessary to prevent a forgery attack where an adversary extends the hash chains forward. WOTS+C (compressed WOTS+) eliminates the transmission of this checksum entirely, achieving a smaller signature size.

In WOTS+, the checksum C is calculated from the message digest D.

$$C = \sum_{i=1}^{l} (w - 1 - d_i)$$

This checksum is then signed using additional chains. In WOTS+C, instead of calculating a checksum for a fixed message digest, the signer modifies the message digest until it possesses a specific, predetermined checksum property. This is done by appending a counter `ctr` and a random salt `R` to the message before hashing. The signer increments `ctr` until the resulting hash digest D satisfies a global target condition.

6.1 The Target Condition

WOTS utilize a parameter `w` to define the number of hashchains and correspondingly their length. The message digest length is `n` bits so `l = 8n/(log2(w))`. The values $d_{i \in [l]}$ are treated as integers in `0...(w-1)`.

Let `Swn` be a protocol parameter (often near the expected value, but may be chosen larger to trade signing/grinding cost for verification cost). If the hash output is uniform, the expected value of a single digit is `d_i = (w-1)/2`. For `l` digits, the expected `Swn` is:

$$S_{wn} = \sum_{i=0}^{l-1} d_i$$

By enforcing this condition during signing (including grinding), the verifier can simply assume the checksum is `Swn` without receiving it. If the reconstructed digest does not sum to `Swn`, the signature is invalid.

Additionally we can manipulate a target condition to decrease the cost of signature verification. The signature generation and verification operations together consist of `St = (w-1)*l` hashing operations. So we can increase `Swn` (grinding complexity) and receive the reduced verification complexity of `(St - Swn)` hashes. Concrete parameters are presented in Section 4.

The general algorithm for base-w representation is following:

6.2 Base-w Representation

WOTS represents a message digest in base `w` so that each digit is an integer in `[0...w-1]`. These digits directly determine the number of hashes to apply to each WOTS chain element during signing and verification. The `base_w()` function converts a byte string into an array of `out_len` base-w digits (this requires `w` to be a power of two.).

```
Algorithm: base_w(m, w, out_len)
    m: byte string
    w: Winternitz parameter
    out_len: output length
Output:
    b_w: array of out_len integers in [0, w-1]

1. in_idx ← 0
2. bits ← 0
3. total ← 0
4. b_w ← []
5. for i from 0 to out_len - 1:
    a. if bits == 0:
        total ← m[in_idx]
        in_idx ← in_idx + 1
        bits ← 8
    b. bits ← bits - log2(w)
    c. b_w[i] ← (total >> bits) AND (w - 1)
6. return b_w
```

But we provide optimized versions for `w = 256, 4` proposed by this specification. For `w=256`:

```
Algorithm: base_w256(m, out_len)
Input:
    m: byte string
    out_len: output length
Output:
    b_w: array of out_len integers in [0, 255]

1. b_w ← []
2. for i from 0 to out_len - 1:
    a. b_w[i] ← m[i]
3. return b_w
```

And for `w=4`:

```
Algorithm: base_w4(m, out_len)
Input:
    m: byte string
    out_len: number of base-4 digits
Output:
    b_w: array of out_len integers in [0,3]

1. b_w ← []
2. for j from 0 to (out_len/4) - 1:
    a. x ← m[j]
    b. b_w[4*j + 0] ← (x >> 6) AND 3
    c. b_w[4*j + 1] ← (x >> 4) AND 3
    d. b_w[4*j + 2] ← (x >> 2) AND 3
    e. b_w[4*j + 3] ← x AND 3
3. return b_w
```

6.3 Chain Function

WOTS derives signature components and public key material by iteratively applying the tweakable hash function along a Winternitz chain. The function `chain(m, start, steps, PK.seed, ADRS)` takes an n-byte input value `m` and applies `Tw(PK.seed, ADRS, ·)` exactly `steps` times, beginning at chain position `start`. For each iteration, the address field hash inside `ADRS` is set to the current chain position `i` (from `start` to `start+steps-1`) to ensure domain separation between different chain steps. The output is the resulting n-byte value after advancing the chain by `steps`, which is used both for producing signature chain values (advancing from 0 to `msg[i]`) and for reconstructing end-of-chain values during verification (advancing from `msg[i]` to `w-1`).

```
Algorithm: chain(m, start, steps, PK.seed, ADRS)
Input:
    m: input
    start: starting index in chain
    steps: number of iterations
    PK.seed: public seed
    ADRS: tweak
Output:
    n-byte chain output

1. tmp ← m
2. for i from start to start + steps - 1:
    a. setHashAddress(ADRS, i)
    b. tmp ← Tw(PK.seed, ADRS, tmp)
3. return tmp
```

6.4 Key Generation

To create a WOTS public key for a given `keypair` index, we derive one secret starting value per chain using `PRF`, then advance each value to the end of its chain with `chain(..., w-1)`. The resulting array of end-of-chain values is compressed to a fixed-size WOTS public key for that `keypair`.

```
Algorithm: wots_PKgen(SK.seed, PK.seed, ADRS, keypair)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
    keypair: keypair index for this WOTS+ instance
Output:
    pk: compressed WOTS+C public key (n bytes)

1. tmp ← []
2. for i from 0 to l - 1:
    a. setTypeAndClear(ADRS, PRF)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
    d. sk_i ← PRF(PK.seed, ADRS, SK.seed)
    e. setTypeAndClear(ADRS, SF_WOTS_CHAIN)
    f. setKeyPairAddress(ADRS, keypair)
    g. setChainAddress(ADRS, i)
    h. tmp[i] ← chain(sk_i, 0, w - 1, PK.seed, ADRS)
3. setTypeAndClear(ADRS, SF_TREE)
4. setKeyPairAddress(ADRS, keypair)
5. pk ← Tw(PK.seed, ADRS, tmp[0] || ... || tmp[l-1])
6. return pk
```

6.5 Messsage Digest Grinding

WOTS+C enforces the checksum/target condition by searching over the counter `ctr`. The value `R` (a randomness/nonce input to message hashing) may be fixed and need not be resampled during grinding. The function `wots_grind(ADRS, R, PK.seed, PK.root, m)` increments `ctr` and recomputes the message digest until the derived base-w digits satisfy the target condition. It outputs the found `ctr` and the corresponding digest (and `R` is carried in the signature only if verification requires it to recompute the same digest).

Oleks: To consider if we need to mention counter in Section 4.5

```
Algorithm: wots_grind(m, R, PK.seed, PK.root, ADRS)
Input:
    m: message to sign
    R: randomness
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    msg: array of l integers
    ctr: final counter value used

1. setTypeAndClear(ADRS, H_MSG)
2. for ctr from 0 to 2^32 - 1:
    a. digest ← H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
    b. msg ← base_w(digest, l)
    c. sum ← 0
    d. for i from 0 to l - 1:
```

```
        sum ← sum + msg[i]
    e. if sum == Swn:
        return (msg, ctr)
3. abort with error  // R must be resampled
```

Oleks: to add expected counter value for different approaches

Additionally we need to define a func which allows to calculate the digest without grinding (by providing the `ctr` value directly).

```
Algorithm: wots_digest(m, R, ctr, PK.seed, PK.root, ADRS)
Input:
    m: message
    R: randomness from signature
    ctr: counter
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    msg: array of l integers
    valid: boolean indicating if sum == Swn

1. setTypeAndClear(ADRS, H_MSG)
2. digest ← H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
3. msg ← base_w(digest, l)
4. sum ← 0
5. for i from 0 to l - 1:
    sum ← sum + msg[i]
6. valid ← (sum == Swn)
7. return (msg, valid)
```

6.6 Signature Generation

To sign, WOTS+C instance computes the message digits and then reveals, for each chain, the chain value after exactly `msg[i]` steps (starting from the secret-derived chain `start`). These revealed values form the WOTS signature vector. The signature also carries the randomness `R` and grinding counter `ctr` needed for the verifier to reproduce the same message digits.

```
Algorithm: wots_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, keypair)
Input:
    m: message
    SK.seed: secret seed
    SK.prf: PRF key for randomness
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
    keypair: keypair index for this WOTS+ instance
Output:
    sig: WOTS+C signature (R || ctr || chain_values)

1. opt_rand ← random(n) or PK.seed  // for randomized or deterministic signing
2. R ← PRF_msg(SK.prf, PK.seed, opt_rand, m)
3. (msg, ctr) ← wots_grind(m, R, PK.seed, PK.root, ADRS)
4. sig ← R || toByte(ctr, c)
5. for i from 0 to l - 1:
    a. setTypeAndClear(ADRS, PRF)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
```

```
    d. sk_i ← PRF(PK.seed, ADRS, SK.seed)
    e. setTypeAndClear(ADRS, SF_WOTS_CHAIN)
    f. setKeyPairAddress(ADRS, keypair)
    g. setChainAddress(ADRS, i)
    h. sig ← sig || chain(sk_i, 0, msg[i], PK.seed, ADRS)
6. return sig
```

6.7 Signature Verification (Public Key Recovery)

Verification reconstructs the WOTS+C public key implied by the signature. For each chain element, the verifier continues hashing forward from the signature's provided intermediate value for `w-1-msg[i]` steps to reach the end-of-chain value. Compressing all end-of-chain values yields the recovered WOTS+C public key, which can then be checked against the expected value in the surrounding scheme.

```
Algorithm: wots_pkFromSig(sig, m, PK.seed, PK.root, ADRS, keypair)
Input:
    sig: WOTS+C signature vector
    m: signed message
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
    keypair: keypair index for this WOTS+ instance
Output:
    pk: recovered public key (n bytes), or $\bot$ if invalid

1. R ← sig[0:R_SIZE]
2. ctr ← sig[R_SIZE:R_SIZE + c]
3. chains_start ← R_SIZE + c
4. (msg, valid) ← wots_digest(m, R, ctr, PK.seed, PK.root, ADRS)
5. if not valid:
    return $\bot$
6. tmp ← []
7. for i from 0 to l - 1:
    a. setTypeAndClear(ADRS, SF_WOTS_CHAIN)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
    d. sig_i ← sig[chains_start + i*n : chains_start + (i+1)*n]
    e. tmp[i] ← chain(sig_i, msg[i], w - 1 - msg[i], PK.seed, ADRS)
8. setTypeAndClear(ADRS, SF_TREE)
9. setKeyPairAddress(ADRS, keypair)
10. pk ← Tw(PK.seed, ADRS, tmp[0] || ... || tmp[l-1])
11. return pk
```

7 UXMSS. The Stateful Tree

Standard XMSS uses a balanced binary tree, which minimizes the maximum path length but imposes a logarithmic cost on every signature. For `N` signatures, every signature has size proportional to `log_2(N)`. SHRINCS utilizes an Unbalanced Merkle Tree (specifically a "Right-Skewed" tree) to optimize for the typical Bitcoin use case: a wallet that performs very few transactions in its lifecycle (we are targeting `N<=10`).

In this topology the `q`-th signature requires a path of length `q`. This linear growth is superior to logarithmic growth for small `q`. For `q=1`, the path is just 16 bytes. Even for `q=10`, the path is $10 \times 16 = 160$ bytes, which is extremely small compared to the fixed ≈ 2.5KB of a balanced tree or the ≈ 8KB of SLH-DSA.

7.1 UXMSS Tree Structure

Oleks: I hope it's not confusing we have layers numeration different for stateless and statefull instance
The UXMSS tree is constructed recursively as follows:

- Level 0 is the root public key `PK.sf`.
- Level `i` (internal):
  - Left child: WOTS+C public key for signature `q=i`
  - Right child: Root of subtree for signatures `q>i`
- Level `hsf` (bottom)
  - Left child: WOTS+C public key for signature `q=hsf`
  - Right child: WOTS+C public key for signature `q=hsf+1`

7.2 UXMSS Authentication Path

For the `q`-th signature (`q` = count after signing), the authentication path contains `q` nodes:

```
auth_path = [node_0, node_1, ..., node_q-1]
```

Root computation from auth path:

```
Algorithm: root_from_auth(pk, auth, q, PK.seed, ADRS)
Input:
    pk: WOTS+C public key for signature q
    auth: authentication path [auth[0], ..., auth[q-1]] from bottom to top
    q: signature number
    PK.seed: public seed
    ADRS: tweak
Output:
    root: computed tree root (PK.sf)

1. node ← pk
2. setTypeAndClear(ADRS, SF_TREE)
3. if q <= hsf:
    for i from 0 to q - 1:
        a. level ← q - i
        b. setTreeHeight(ADRS, level)
        c. setTreeIndex(ADRS, 0)
        d. if i == 0:
            node ← Tw(PK.seed, ADRS, node || auth[i])
        e. else:
            node ← Tw(PK.seed, ADRS, auth[i] || node)
4. else:  // q = hsf + 1
    for i from 0 to hsf - 1:
        a. level ← hsf - i
        b. setTreeHeight(ADRS, level)
        c. setTreeIndex(ADRS, 0)
        d. if i == 0:
            node ← Tw(PK.seed, ADRS, auth[i] || node)
        e. else:
            node ← Tw(PK.seed, ADRS, auth[i] || node)
5. return node
```

8 FORS+C

FORS (Forest of Random Subsets) is a few-time signature scheme used at the bottom of the stateless hypertree. FORS allows to sign a limited number of messages, when the security degrades with each use. Standard FORS signs a message by revealing `k` secret values, one from each of `k` binary trees of height `a`. The message digest determines which leaf to reveal from each tree. FORS+C applies the same checksum grinding optimization as WOTS+C to reduce signature size.

8.0.1 FORS Structure

FORS consists of `k` binary trees, each of height `a`:

- Each tree has `2â` leaves
- Each leaf is derived from `SK.seed` using PRF
- The message digest is split into `k` chunks of `a` bits each
- Each chunk selects one leaf index from the corresponding tree

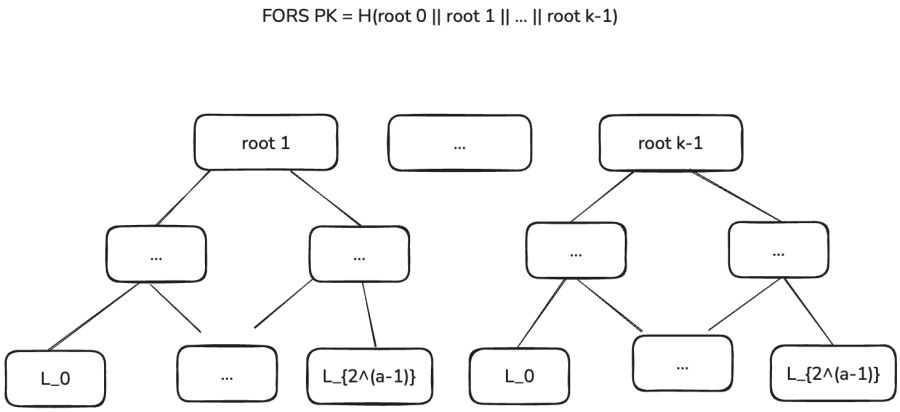

Figure 2: FORS architecture

8.0.2 Parameters

Parameter	SHRINCS-SHA2-128s
`n`	128 bits (16 bytes)
`k`	6
`a`	22

8.0.3 Grinding Condition

```
digest = H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
condition: digest[last a bits] == 0
```

Cost is expected to be $\approx 2^a$ iterations.

8.0.4 Message to Indices

```
Algorithm: fors_msg_to_indices(digest)
Input:
    digest: message digest
Output:
```

```
    indices: array of k integers in [0, 2^a - 1]

1. indices ← []
2. for i from 0 to k - 1:
    a. indices[i] ← extract_bits(digest, i × a, a)
3. return indices
```

8.0.5 Grinding

```
Algorithm: fors_grind(m, R, PK.seed, PK.root, ADRS)
Input:
    m: message to sign
    R: randomness
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    indices: array of k integers
    ctr: counter value

1. setTypeAndClear(ADRS, H_MSG)
2. for ctr from 0 to 2^32 - 1:
    a. digest ← H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
    b. indices ← fors_msg_to_indices(digest)
    c. if indices[k-1] == 0:
        return (indices, ctr)
3. abort with error
```

8.0.6 Secret Key Generation

```
Algorithm: fors_SKgen(SK.seed, PK.seed, ADRS, tree_idx, leaf_idx)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
    tree_idx: tree index (0 to k-1)
    leaf_idx: leaf index within tree
Output:
    sk: secret value (n bytes)

1. setTypeAndClear(ADRS, PRF)
2. setKeyPairAddress(ADRS, tree_idx)
3. setTreeIndex(ADRS, leaf_idx)
4. sk ← PRF(PK.seed, ADRS, SK.seed)
5. return sk
```

8.0.7 TreeHash

```
Algorithm: fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, target_height, start_idx)
Input:
    SK.seed, PK.seed: seeds
    ADRS: tweak
    tree_idx: tree index (0 to k-1)
    target_height: height of output node
    start_idx: leftmost leaf index in subtree
Output:
    node: hash value (n bytes)
```

```
1. if target_height == 0:
    a. sk ← fors_SKgen(SK.seed, PK.seed, ADRS, tree_idx, start_idx)
    b. setTypeAndClear(ADRS, SL_FORS)
    c. setKeyPairAddress(ADRS, tree_idx)
    d. setTreeHeight(ADRS, 0)
    e. setTreeIndex(ADRS, start_idx)
    f. return Tw(PK.seed, ADRS, sk)
2. left ← fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, target_height-1, start_idx)
3. right ← fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, target_height-1, start_idx +
           2^(target_height-1))
4. setTypeAndClear(ADRS, SL_FORS)
5. setKeyPairAddress(ADRS, tree_idx)
6. setTreeHeight(ADRS, target_height)
7. setTreeIndex(ADRS, start_idx >> target_height)
8. return Tw(PK.seed, ADRS, left || right)
```

8.0.8 FORS Authentication path

```
Algorithm: fors_auth_path(SK.seed, PK.seed, ADRS, tree_idx, leaf_idx)
Input:
    SK.seed, PK.seed: seeds
    ADRS: tweak
    tree_idx: tree index (0 to k-1)
    leaf_idx: leaf index being revealed
Output:
    auth: authentication path (a × n bytes)

1. auth ← []
2. for h from 0 to a - 1:
    a. sibling_start ← (leaf_idx XOR 2^h) AND (0xFFFFFFFF << h)
    b. auth ← auth || fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, h, sibling_start)
3. return auth
```

8.0.9 Signature generation

```
Algorithm: fors_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS)
Input:
    m: message
    SK.seed: secret seed
    SK.prf: PRF key
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    sig: FORS+C signature

1. opt_rand ← random(n) or PK.seed
2. R ← PRF_msg(SK.prf, PK.seed, opt_rand, m)
3. (indices, ctr) ← fors_grind(m, R, PK.seed, PK.root, ADRS)
4. sig ← R || toByte(ctr, c)
5. for i from 0 to k - 2:
    a. sk_i ← fors_SKgen(SK.seed, PK.seed, ADRS, i, indices[i])
    b. auth_i ← fors_auth_path(SK.seed, PK.seed, ADRS, i, indices[i])
    c. sig ← sig || sk_i || auth_i
6. sk_last ← fors_SKgen(SK.seed, PK.seed, ADRS, k-1, 0)
```

```
7. root_last ← fors_treehash(SK.seed, PK.seed, ADRS, k-1, a, 0)
8. sig ← sig || sk_last || root_last
9. return sig
```

8.0.10 Public Key Recovery

```
Algorithm: fors_pkFromSig(sig, m, PK.seed, PK.root, ADRS)
Input:
    sig: FORS+C signature
    m: message
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    pk: FORS public key (n bytes), or $\bot$ if invalid

1. R ← sig[0 : R_SIZE]
2. ctr ← sig[R_SIZE : R_SIZE + c]
3. setTypeAndClear(ADRS, H_MSG)
4. digest ← H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
5. indices ← fors_msg_to_indices(digest)
6. if indices[k-1] != 0:
    return $\bot$

7. roots ← []
8. offset ← R_SIZE + c
9. for i from 0 to k - 2:
    a. sk_i ← sig[offset : offset + n]
    b. auth_i ← sig[offset + n : offset + n + a × n]
    c. offset ← offset + n + a × n
    d. setTypeAndClear(ADRS, SL_FORS)
    e. setKeyPairAddress(ADRS, i)
    f. setTreeHeight(ADRS, 0)
    g. setTreeIndex(ADRS, indices[i])
    h. node ← Tw(PK.seed, ADRS, sk_i)
    i. idx ← indices[i]
    j. for h from 0 to a - 1:
        setTreeHeight(ADRS, h + 1)
        setTreeIndex(ADRS, idx >> 1)
        if (idx AND 1) == 0:
            node ← Tw(PK.seed, ADRS, node || auth_i[h × n : (h+1) × n])
        else:
            node ← Tw(PK.seed, ADRS, auth_i[h × n : (h+1) × n] || node)
        idx ← idx >> 1
k. roots[i] ← node
10. sk_last ← sig[offset : offset + n]
11. root_last ← sig[offset + n : offset + 2 × n]
12. roots[k-1] ← root_last
13. setTypeAndClear(ADRS, SL_FORS)
14. setTreeHeight(ADRS, a)
15. setTreeIndex(ADRS, 0)
16. pk ← Tw(PK.seed, ADRS, roots[0] || ... || roots[k-1])
17. return pk
```

9 SHRINCS

9.0.1 Key Generation

```
Algorithm: SHRINCS.KeyGen()
Output:
    SK: secret key
    PK: public key
    state: initial state

1. seed ←$ {0,1}^384
2. SK.seed ← seed[0:16]
3. SK.prf ← seed[16:32]
4. PK.seed ← seed[32:48]
5. ADRS ← new_ADRS()

// Generate stateful public key (UXMSS root)
6. setLayerAddress(ADRS, 0)
7. setTreeAddress(ADRS, 0)
8. PK_sf ← compute_initial_stateful_root(SK.seed, PK.seed, ADRS)

// Generate stateless public key (SPHINCS+C root)
9. PK_sl ← sphincs_root(SK.seed, PK.seed, ADRS)

// Combined params
10. PK.root ← H(0x08 || PK.seed || PK_sf || PK_sl)
11. SK ← (SK.seed, SK.prf, PK.seed, PK.root, PK_sf, PK_sl)
12. PK ← (PK.seed, PK.root)
13. state ← { q: 0, valid: true }
14. return (SK, PK, state)
```

9.0.2 Recovery from Seed

```
Algorithm: SHRINCS.Restore(seed)
Input:
    seed: master seed
Output:
    SK: secret key
    PK: public key
    state: state "LOST"

1. SK.seed ← seed[0:16]
2. SK.prf ← seed[16:32]
3. PK.seed ← seed[32:48]
4. ADRS ← new_ADRS()
5. PK_sf ← compute_max_stateful_root(SK.seed, PK.seed, ADRS)
6. PK_sl ← sphincs_root(SK.seed, PK.seed, ADRS)
7. PK.root ← H_n(0x08 || PK.seed || PK_sf || PK_sl)
8. SK ← (SK.seed, SK.prf, PK.seed, PK.root, PK_sf, PK_sl)
9. PK ← (PK.seed, PK.root)
10. state ← { q: $\bot$, valid: false }
11. return (SK, PK, state)
```

9.1 Statefull signing

9.2 Stateless signing

9.3 Statefull verification

9.4 Stateless verification

10 Data structures

10.1 Secret Key Format

10.2 Public Key Format

10.3 Stateful Signature Format

10.4 Stateless Signature Format